VSAM Key and Duplicate-Key Tests

COBOL • Logic-level simulation • Batch rollback verification

1. Aim

To test unique account-key validation, duplicate-key detection, preservation of leading zeros, and rollback of a batch when any required operation fails. This program simulates a VSAM-like record set in memory; it does not access a real VSAM dataset.

2. Problem Statement

A batch contains multiple account insert operations that must be treated as one logical unit. If any required operation fails before the batch is committed, all changes made by that batch must be undone. The test must print each operation's result and the final record count. Inserting an account ID that already exists must produce a DUPLICATE KEY result.

Test setup: Start with account IDs 00001, 00002 and 00003. Insert 00004 successfully, try 00002 as a duplicate, then test a batch that inserts 00005 followed by duplicate 00003. The batch should roll back, leaving the final record count at 4.

3. Algorithm

1. Initialise the simulated account table with keys 00001, 00002 and 00003.
2. Display the initial record count.
3. For each insert, search existing records for the requested key.
4. If the key exists, display DUPLICATE KEY and do not change the record count.
5. If the key does not exist and capacity remains, append the key and display INSERT SUCCESS.
6. Before the batch test, copy the table and record count as a rollback snapshot.
7. Insert 00005, then attempt to insert existing key 00003.
8. If any batch operation fails, restore the snapshot and display BATCH ROLLBACK SUCCESS.
9. Display the final record count.

4. COBOL Code

```
IDENTIFICATION DIVISION.
PROGRAM-ID. VSAM-KEY-TEST.

DATA DIVISION.
WORKING-STORAGE SECTION.

01 ACCOUNT-TABLE.
   05 ACCOUNT-ID OCCURS 10 TIMES PIC X(5).
01 ACCOUNT-COUNT PIC 99 VALUE 3.
01 BACKUP-TABLE.
   05 BACKUP-ID OCCURS 10 TIMES PIC X(5).
01 BACKUP-COUNT PIC 99 VALUE 0.
01 NEW-ACCOUNT PIC X(5).
01 FOUND-FLAG PIC X VALUE 'N'.
01 OP-STATUS PIC X VALUE SPACE.
01 BATCH-FAILED PIC X VALUE 'N'.
01 I PIC 99 VALUE 0.

PROCEDURE DIVISION.
    MOVE '00001' TO ACCOUNT-ID(1)
    MOVE '00002' TO ACCOUNT-ID(2)
    MOVE '00003' TO ACCOUNT-ID(3)

    DISPLAY 'INITIAL RECORD COUNT: ' ACCOUNT-COUNT

    DISPLAY ' '
    DISPLAY 'TEST 1: INSERT NEW ACCOUNT'
    MOVE '00004' TO NEW-ACCOUNT
    PERFORM INSERT-ACCOUNT

    DISPLAY ' '
    DISPLAY 'TEST 2: INSERT DUPLICATE KEY'
    MOVE '00002' TO NEW-ACCOUNT
    PERFORM INSERT-ACCOUNT

    DISPLAY ' '
    DISPLAY 'TEST 3: BATCH ROLLBACK'
    MOVE ACCOUNT-TABLE TO BACKUP-TABLE
    MOVE ACCOUNT-COUNT TO BACKUP-COUNT
    MOVE 'N' TO BATCH-FAILED

    MOVE '00005' TO NEW-ACCOUNT
    PERFORM INSERT-ACCOUNT
    IF OP-STATUS NOT = 'S'
        MOVE 'Y' TO BATCH-FAILED
    END-IF

    MOVE '00003' TO NEW-ACCOUNT
    PERFORM INSERT-ACCOUNT
    IF OP-STATUS NOT = 'S'
        MOVE 'Y' TO BATCH-FAILED
    END-IF

    IF BATCH-FAILED = 'Y'
        MOVE BACKUP-TABLE TO ACCOUNT-TABLE
        MOVE BACKUP-COUNT TO ACCOUNT-COUNT
        DISPLAY 'BATCH ROLLBACK SUCCESS'
    ELSE
        DISPLAY 'BATCH COMMITTED'
    END-IF

    DISPLAY ' '
    DISPLAY 'FINAL RECORD COUNT: ' ACCOUNT-COUNT
    STOP RUN.

INSERT-ACCOUNT.
    MOVE 'N' TO FOUND-FLAG
    MOVE 'S' TO OP-STATUS

    PERFORM VARYING I FROM 1 BY 1
        UNTIL I > ACCOUNT-COUNT
        IF ACCOUNT-ID(I) = NEW-ACCOUNT
            MOVE 'Y' TO FOUND-FLAG
        END-IF
    END-PERFORM

    IF FOUND-FLAG = 'Y'
        MOVE 'D' TO OP-STATUS
        DISPLAY 'DUPLICATE KEY: ' NEW-ACCOUNT
    ELSE
        IF ACCOUNT-COUNT < 10
            ADD 1 TO ACCOUNT-COUNT
            MOVE NEW-ACCOUNT TO ACCOUNT-ID(ACCOUNT-COUNT)
```

```
                DISPLAY 'INSERT SUCCESS: ' NEW-ACCOUNT
            ELSE
                MOVE 'F' TO OP-STATUS
                DISPLAY 'TABLE FULL'
            END-IF
        END-IF.
```

5. Expected Output

```
INITIAL RECORD COUNT: 03

TEST 1: INSERT NEW ACCOUNT
INSERT SUCCESS: 00004

TEST 2: INSERT DUPLICATE KEY
DUPLICATE KEY: 00002

TEST 3: BATCH ROLLBACK
INSERT SUCCESS: 00005
DUPLICATE KEY: 00003
BATCH ROLLBACK SUCCESS

FINAL RECORD COUNT: 04
```

6. Result

The logic-level test successfully demonstrates insertion of a new key, rejection of a duplicate key, and rollback of the batch after a duplicate-key failure. Account 00005 is removed by restoring the snapshot, so the final count remains 4 (the original three accounts plus 00004). Leading zeros are preserved because account IDs are stored as PIC X(5).

Note: This is an in-memory simulation, not a real VSAM transaction test. A real KSDS test must use the dataset's actual key definition and file status (a duplicate primary key commonly returns status 22). True multi-record rollback requires a transaction/backout facility supported by the execution environment; ordinary COBOL WRITE statements alone do not make a batch atomic.